

K-Means Clustering versione sequenziale e parallela con OpenMP

Confronto AoS vs SoA e analisi delle performance

Alessio Poggesi

Parallel Programming for Machine Learning



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.

Table of Contents

1 Introduzione del Problema

► Introduzione del Problema

► Approcci e Strategie

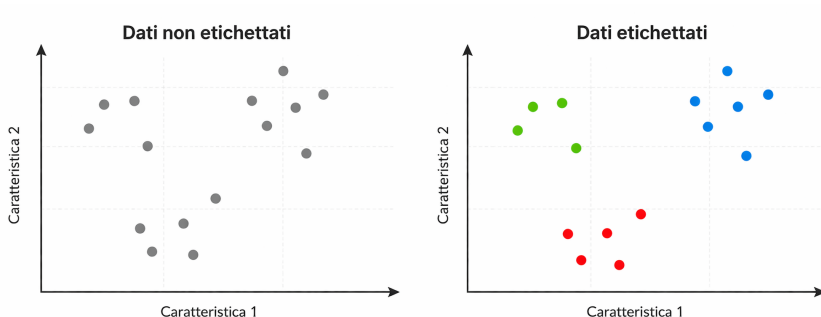
► Risultati

► Conclusione

K-Means: idea generale

1 Introduzione del Problema

- K-Means è un algoritmo di clustering non supervisionato
- Divide N punti in K gruppi
- Ogni gruppo è rappresentato da un centroide
- Obiettivo: minimizzare la distanza dei punti dal proprio cluster



Come funziona K-Means

1 Introduzione del Problema

Fase 1: Assignment

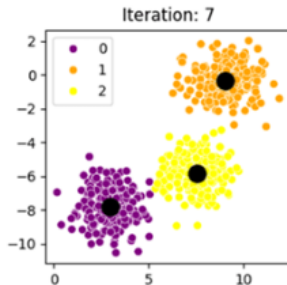
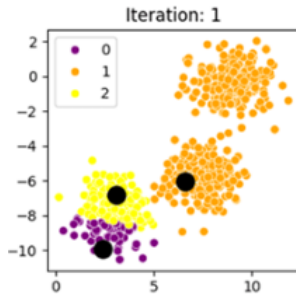
Ogni punto viene assegnato al centroide più vicino

Fase 2: Update

Ogni centroide viene ricalcolato come media dei punti assegnati



Si ripete fino a convergenza



Perché è una sfida computazionale

1 Introduzione del Problema

- Il costo dominante è la fase di **assignment**
- Per ogni iterazione, ogni punto viene confrontato con tutti i centroidi
- Complessità per iterazione: $O(N \cdot K \cdot D)$
- Le prestazioni dipendono anche da memoria, cache e sincronizzazione

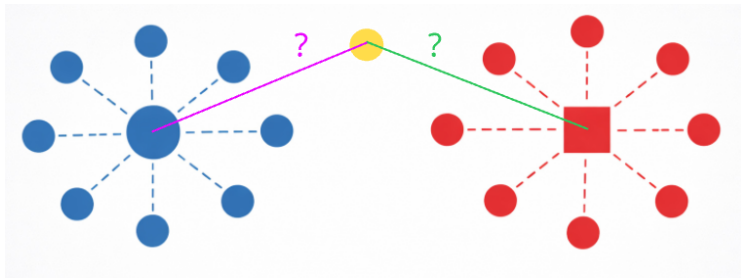


Table of Contents

2 Approcci e Strategie

► Introduzione del Problema

► **Approcci e Strategie**

► Risultati

► Conclusione

Approccio utilizzato

2 Approcci e Strategie

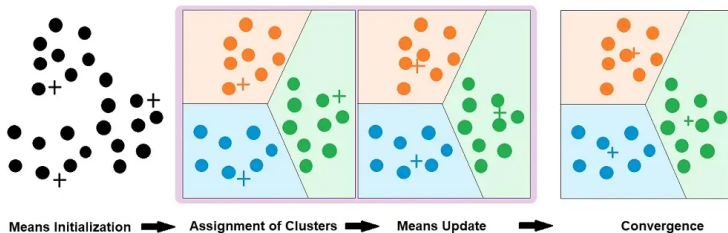
- Implementazione di tre versioni di K-Means:
 - **Sequenziale**: baseline di riferimento
 - **OpenMP AoS**: punti come array di strutture
 - **OpenMP SoA**: coordinate separate in array distinti
- Confronto delle versioni rispetto a:
 - numero di thread
 - scheduling OpenMP
 - chunk size
 - layout dei dati in memoria

Obiettivo: valutare l'impatto della parallelizzazione e del layout memoria sulle prestazioni

Versione sequenziale

2 Approcci e Strategie

- La versione sequenziale è la baseline di riferimento
- Lo stesso schema viene mantenuto anche nelle versioni parallele
- A ogni iterazione:
 - **Assignment:** assegnazione al centroide più vicino
 - **Update:** ricalcolo dei centroidi come media dei punti assegnati
- Distanza euclidea al quadrato
- Arresto per convergenza oppure massimo 600 iterazioni



Parallelizzazione dell'assignment

2 Approcci e Strategie

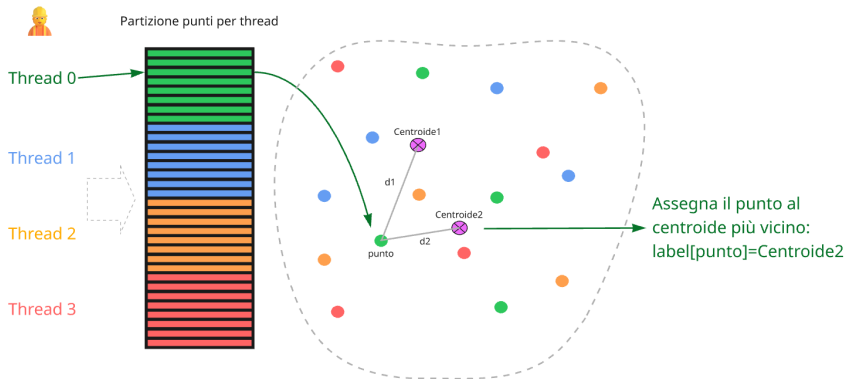
Per ogni punto si calcola la distanza da tutti i centroidi e si assegna al più vicino.

- **Problema:** a ogni iterazione ogni punto deve essere confrontato con tutti i centroidi
- **Osservazione:** i punti sono indipendenti tra loro
- **Scelta adottata:** loop parallelo sui punti con

```
#pragma omp parallel for schedule(runtime)
```

Parallelizzazione dell'assignment

2 Approcci e Strategie



- **Perché funziona:** ogni thread legge i centroidi condivisi e scrive solo in `labels[i]`
È la sezione più naturalmente parallelizzabile dell'algoritmo

Parallelizzazione dell'update

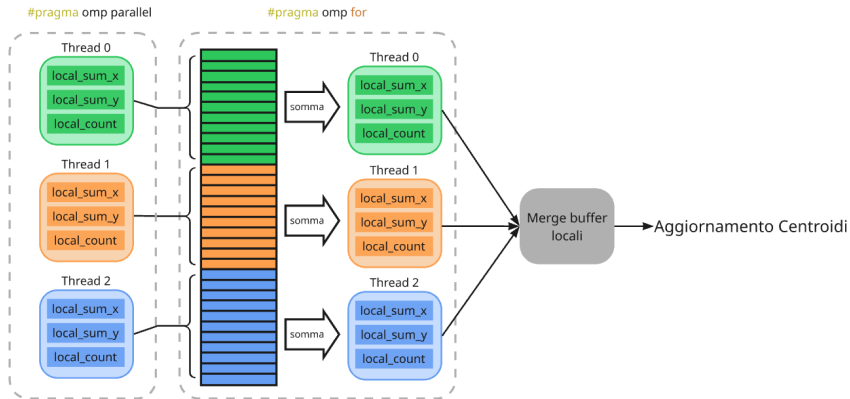
2 Approcci e Strategie

- **Problema:** più punti possono contribuire allo stesso cluster
- Una parallelizzazione diretta introdurrebbe **race condition**
- L'uso di `critical` o `atomic` aumenterebbe molto l'overhead
- **Scelta adottata:** accumulatori locali per thread e merge finale

`parallel` → `buffer locali` → `omp for` → `merge` → `parallel for`

Parallelizzazione dell'update

2 Approcci e Strategie



- **Perché funziona:** Ogni thread accumula solo nei propri buffer locali; la sincronizzazione viene rimandata al merge finale.

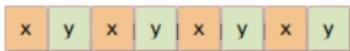
AoS vs SoA: layout dei dati

2 Approcci e Strategie

AoS – Array of Structures

- Ogni punto è una struct (x, y)
- Rappresentazione più naturale
- Accessi interleavati in memoria

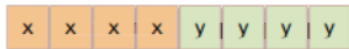
AoS memory layout



SoA – Structure of Arrays

- Coordinate separate in array distinti
- Accessi più regolari
- Layout più favorevole alla località

SoA memory layout



L'algoritmo è lo stesso, cambia solo il layout dei dati in memoria

Setup sperimentale

2 Approcci e Strategie

Macchina e ambiente

- CPU Intel Core i7-11800H
- 16 GB di RAM
- Sistema shared-memory
- Progetto in C++17 con OpenMP
- Build Release con ottimizzazioni

Dataset e parametri

- Dataset sintetici 2D generati con `make_blobs`
- Dimensioni: da 10^4 a 10^6 punti
- Numero di cluster: $K \in \{6, 10, 20\}$
- Thread testati: $\{1, 2, 4, 8, 16, 32\}$
- Scheduling: `static`, `dynamic`
- Chunk size: $\{0, 1, 32, 128\}$

Metodologia di misura e correttezza

2 Approcci e Strategie

- 5 esecuzioni per ogni configurazione
- 1 warm-up run scartato
- Misurata solo la fase di `fit()`
- Stessa inizializzazione dei centroidi tra sequenziale e parallelo
- Raccolta di media, deviazione standard, minimo e massimo

Validazione della correttezza

- Confronto con la baseline sequenziale
- Verifica tramite **SSE** finale
- Controllo anche del numero di iterazioni a convergenza

Table of Contents

3 Risultati

► Introduzione del Problema

► Approcci e Strategie

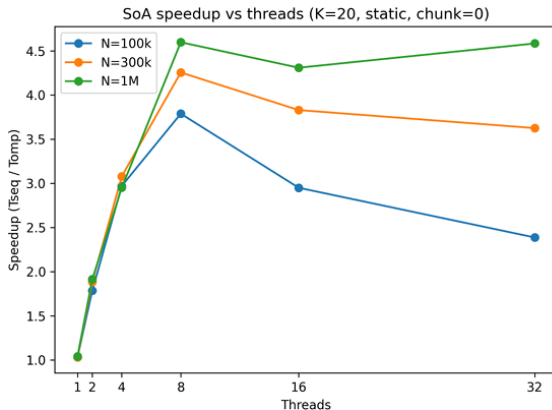
► **Risultati**

► Conclusione

Andamento generale dello speedup

3 Risultati

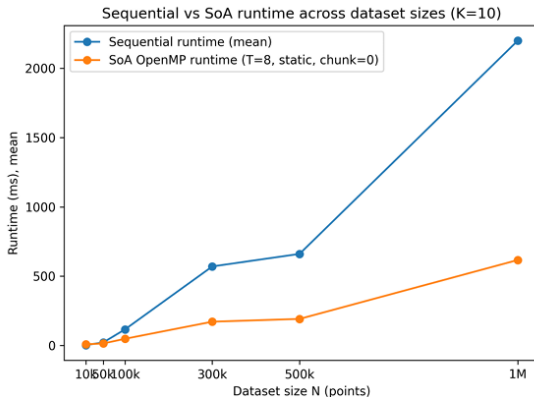
- I dataset piccoli scalano poco
- Lo speedup migliora al crescere della dimensione del problema
- I risultati migliori si osservano sui dataset più grandi
- Il picco viene raggiunto tipicamente intorno a 8 thread



Confronto dei tempi di esecuzione

3 Risultati

- Il vantaggio della versione parallela cresce con la dimensione del dataset
- La variante **SoA** riduce in modo netto il runtime rispetto alla baseline sequenziale
- Sui dataset piccoli il beneficio è limitato, mentre sui dataset grandi diventa evidente



Migliore configurazione e trend dello speedup

3 Risultati

Miglior configurazione

OpenMP SoA

8 thread

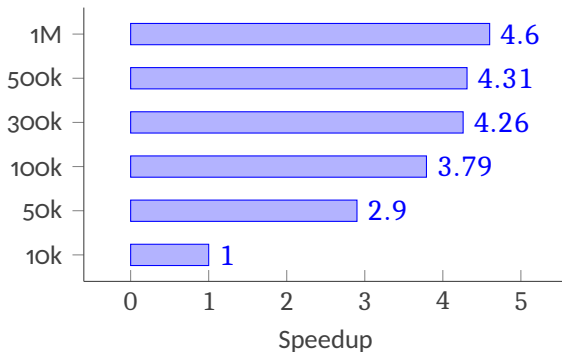
`static, chunk = 0`

$N = 10^6$, $K = 20$

$\approx 4.60\times$

- configurazione più efficace
- migliore sui dataset grandi
- conferma SoA

Best speedup SoA al crescere di N



SoA, $K = 20$, migliore configurazione per ciascun dataset

Osservazioni finali e limiti di scalabilità

3 Risultati

Risultati osservati

- La variante **SoA** è risultata in generale più efficiente di AoS
- Lo scheduling `static` è quasi sempre migliore di `dynamic`
- Lo speedup cresce al crescere della dimensione del problema

Cosa limita la scalabilità

- saturazione della *memory bandwidth*
- costo del merge nell'update dei centroidi
- overhead di scheduling, soprattutto con `dynamic`
- oversubscription ad alti thread count

Interpretazione

Il parallelismo è efficace soprattutto sui problemi grandi, ma i benefici non crescono indefinitamente: oltre una certa soglia, overhead e limiti di memoria riducono la scalabilità.

Table of Contents

4 Conclusione

► Introduzione del Problema

► Approcci e Strategie

► Risultati

► Conclusione

Riepilogo e Conclusioni

4 Conclusione

- Sono state implementate tre versioni di K-Means:
 - baseline sequenziale
 - OpenMP AoS
 - OpenMP SoA
- La correttezza è stata verificata confrontando la SSE finale con la versione sequenziale
- La configurazione migliore è risultata:
 - **OpenMP SoA**
 - `static, chunk = 0`
 - 8 thread
 - speedup massimo $\approx 4.60\times$
- Il layout dei dati e la strategia di parallelizzazione hanno avuto un impatto concreto sulle prestazioni

K-Means Clustering versione sequenziale e parallela con OpenMP

Thank you for listening!
Any questions?